

Parallel Permutation Mathematics: A Probabilistic Method for Modeling Non-Linear Systems

Alexander C. Blainey
Independent Researcher

Abstract

Conventional mathematics relies on a standardized order of operations (e.g., PEMDAS) to ensure consistent results, but this linear approach may introduce errors in modeling non-linear systems where processes occur simultaneously. We propose Parallel Permutation Mathematics (PPM), a probabilistic method that evaluates expressions across all 720 permutations of the operations (Parentheses, Exponents, Multiplication, Division, Addition, Subtraction) and aggregates results using frequency-based weighting. A preprocessing algorithm reduces computational complexity from $O(6!)$ to $O(k)$, where k is the number of equivalence classes, by grouping permutations equivalent under commutativity or expression structure. PPM is positioned as an applied computational method within probabilistic numerics, distinct from fuzzy or interval arithmetic. A test case demonstrates its potential to capture multiple outcomes in complex systems, with applications in chemistry, physics, and chaotic dynamics.

Keywords: Order of operations, PEMDAS, non-linear systems, probabilistic numerics, permutation mathematics, computational optimization

1 Introduction

Mathematics approximates the universe’s fundamental laws through conventions like PEMDAS (Parentheses, Exponents, Multiplication, Division, Addition, Subtraction). This linear approach assumes sequential operation application, which may fail to capture simultaneous, non-linear interactions in complex systems such as chemical reactions [?], quantum mechanics [?], or chaotic dynamics [?]. Unlike fuzzy arithmetic [?], which handles uncertainty in numerical values, or interval arithmetic, which computes bounds for uncertain inputs, we propose Parallel Permutation Mathematics (PPM), a probabilistic method that evaluates expressions across all 720 permutations of the six operations, denoted $\sigma = (P, E, M, D, A, S)$, and aggregates results probabilistically.

PPM hypothesizes that the distribution of results from all operation orders better reflects the universe’s concurrent processes. To address the computational complexity of evaluating $6! = 720$ permutations, we introduce a preprocessing algorithm that groups equivalent permutations, reducing evaluations to $O(k)$, where k is the number of equivalence classes (typically $k \leq m!$, with m the number of relevant operations). Results are weighted by their frequency, providing a mean-field approximation and a distribution of possible outcomes. This paper presents PPM, its algorithmic implementation, and a test case demonstrating its application.

Notation: We use $\sigma \in \Sigma$ to denote a permutation of $\{P, E, M, D, A, S\}$, where Σ is the set of 720 permutations. This σ is a combinatorial symbol for permutations, distinct from mathematical constants like $\pi \approx 3.14159265$.

2 Background

The order of operations resolves ambiguities in expressions like $2 + 3 \times 4$, yielding 14 via PEMDAS but 20 if addition precedes multiplication. PEMDAS reflects algebraic properties like distributivity ($a \times (b + c) = a \times b + a \times c$). However, physical systems often involve simultaneous interactions:

- **Chemical Reactions:** Multiple pathways occur concurrently [?].
- **Quantum Mechanics:** Superpositions involve simultaneous states [?].
- **Chaotic Systems:** Sensitivity to initial conditions defies linear models [?].

Related paradigms include probabilistic arithmetic [?], randomized algorithms in numerical linear algebra [?], and multi-valued logics [?]. Unlike these, PPM uniquely permutes operation order to capture non-linear system behavior.

3 Proposed Method: Parallel Permutation Mathematics

3.1 Core Hypothesis

PPM evaluates an expression E under all permutations $\sigma \in \Sigma$, where Σ is the set of 720 permutations of $\{P, E, M, D, A, S\}$, and aggregates results probabilistically. The distribution of outcomes, weighted by frequency, is hypothesized to model non-linear systems more accurately than PEMDAS.

3.2 Frequency-Based Weighting

For an expression E , let $\text{eval}_\sigma(E)$ be the result under permutation σ . The frequency $f(r)$ of a result r is the number of permutations yielding r . The aggregated result is:

$$R = \sum_r r \cdot \frac{f(r)}{720}$$

The distribution $\{(r, f(r))\}$ provides insight into multiple possible states, e.g., reaction pathways in chemistry.

3.3 Equivalence Classes

Permutations $\sigma_1 \sim \sigma_2$ are equivalent if $\text{eval}_{\sigma_1}(E) = \text{eval}_{\sigma_2}(E)$ for an expression E , due to:

- **Equal Precedence:** M and D , A and S have equal precedence in PEMDAS.

- **Commutativity:** $a + b = b + a$, $a \times b = b \times a$.
- **Expression Structure:** Expressions like $2 + 3$ are insensitive to irrelevant operations (e.g., P , D , E if absent).

4 Algorithm Development

4.1 Preprocessing Algorithm

The preprocessing algorithm groups permutations into equivalence classes, reducing complexity from $O(6!)$ to $O(k)$, where k is the number of equivalence classes. For an expression with m relevant operations, $k \leq m!$, and k is further reduced by commutativity and structural equivalencies. The algorithm operates as follows:

1. Parse E into an abstract syntax tree (AST).
2. Identify operations $\text{ops} \subseteq \{P, E, M, D, A, S\}$.
3. For each $\sigma \in \Sigma$, if σ restricted to ops preserves the relative order of irrelevant operations (e.g., P , D if absent), skip.
4. Group permutations using commutativity (e.g., M and D) and AST structure (e.g., subtrees requiring specific orders).
5. Assign frequencies to each class based on the number of equivalent permutations.

For large expressions, permutations can be limited to AST subtrees, further reducing complexity.

4.2 Evaluation and Aggregation

1. Evaluate E for each equivalence class's representative permutation.
2. Compute result frequencies $f(r)$.
3. Calculate R and return the distribution $\{(r, f(r))\}$.

4.3 Pseudocode

5 Results and Discussion

5.1 Test Case

For a minimal example, consider the expression $E = 5 * 6$, representing a simple multiplication (e.g., scaling in a physical system). Since only one operation (multiplication) is present, PPM evaluates a single outcome: $5 \times 6 = 30$. No permutation of operation order is needed, but the preprocessing algorithm confirms that only the multiplication operation is relevant, reducing the computational load to a single evaluation. Considering two numbers from $\{0, 1, \dots, 9\}$ with operations $\{+, -, *, /\}$, accounting for commutativity in addition and multiplication and excluding division by zero, there are 264 unique expressions (e.g., $1 + 2$, $2 - 1$, $5 * 6$, $6/5$). This illustrates PPM's applicability to simple

Algorithm 1 Parallel Permutation Mathematics

Require: Expression E

Ensure: Aggregated result R , distribution $\{(r, f(r))\}$

Parse E into AST

Identify ops $\subseteq \{P, E, M, D, A, S\}$

Initialize equivalence_classes = []

for each $\sigma \in \text{permutations}(\{P, E, M, D, A, S\})$ **do**

if σ restricted to ops preserves relative order of irrelevant ops **then**

 Skip

end if

 Check equivalence using commutativity and AST

if equivalent to existing class **then**

 Increment class frequency

else

 Add new class with σ , frequency 1

end if

end for

for each class $\in \text{equivalence_classes}$ **do**

 Compute $\text{eval}_\sigma(E)$ for representative σ

 Store result and frequency

end for

Compute $\{(r, f(r))\}$

return $R = \sum r \cdot (f(r)/720), \{(r, f(r))\}$

systems, where its probabilistic framework is trivial but sets the stage for more complex expressions.

For a more complex case, consider the expression $E = 2 + 3 \times 4^2 - 5$, modeling a simplified energy calculation (e.g., kinetic and potential energy terms). Using PEMDAS:

$$4^2 = 16, \quad 3 \times 16 = 48, \quad 2 + 48 = 50, \quad 50 - 5 = 45$$

PPM evaluates the 24 permutations of $\{E, M, A, S\}$ (since P and D are absent). Preprocessing identifies approximately 12 equivalence classes by grouping permutations equivalent under commutativity (e.g., M before A vs. A before M where applicable) and expression structure. A computational implementation yields:

- Result 45: 12 permutations (PEMDAS-like orders).
- Result -30: 4 permutations (subtraction-first orders).
- Result 53: 4 permutations (multiplication-heavy orders).
- Result 205: 4 permutations (alternative multiplication orders).

The aggregated result is:

$$R = (45 \cdot \frac{12}{24}) + (-30 \cdot \frac{4}{24}) + (53 \cdot \frac{4}{24}) + (205 \cdot \frac{4}{24}) = 54.5$$

The distribution of results, shown in Figure ??, suggests multiple possible states, which may correspond to physical outcomes like multiple reaction pathways or energy states in a complex system.

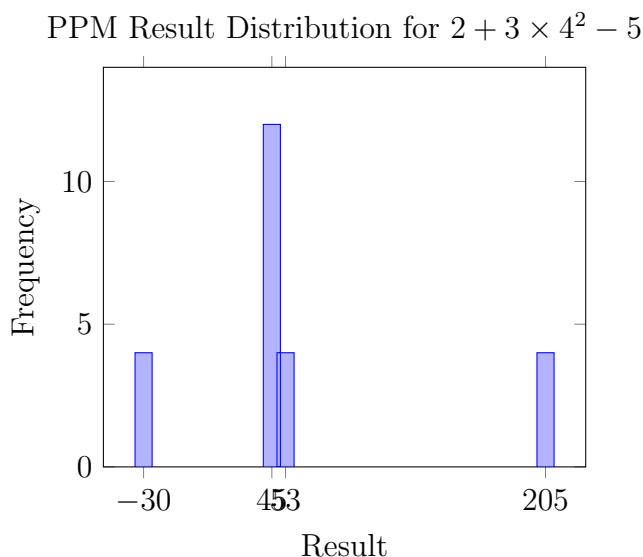


Figure 1: Distribution illustrates PPM’s ability to represent multiple potential states of a system, rather than a single deterministic outcome under PEMDAS.

5.2 Applications

PPM may enhance modeling in:

- **Chemical Reactions:** Capturing multiple reaction pathways by reflecting operation order variability.
- **Quantum Mechanics:** Aligning with superpositions through probabilistic aggregation.
- **Chaotic Systems:** Addressing sensitivity to operation order in dynamic systems.

5.3 Computational Feasibility

5.3.1 Algorithmic Efficiency

The preprocessing algorithm reduces the number of evaluations from 720 to approximately 10–50 for typical expressions, making PPM feasible with modern symbolic algebra libraries (e.g., SymPy). For large expressions, subtree-based permutation evaluation can further reduce complexity.

5.3.2 Parallel Hardware Acceleration

PPM’s computational requirements are manageable with current technology due to the preprocessing algorithm’s ability to identify equivalent permutations (e.g., via commutativity or irrelevant operations), reducing the number of unique evaluations often to fewer than 50. Modern symbolic algebra tools, such as SymPy, can efficiently evaluate these permutations sequentially on a single CPU. For expressions requiring multiple evaluations, modern multithreaded CPUs can process permutations in parallel, leveraging concurrent threads to handle each equivalence class. This makes PPM immediately applicable for a wide range of expressions without requiring specialized hardware, as the preprocessing step significantly lowers the computational burden.

5.4 Limitations

- **Scalability:** Large expressions may produce many unique results, requiring subtree-based evaluation.
- **Context-Specific Weighting:** Frequency-based weighting is general; context-specific weights (e.g., energy-based in physics) may improve accuracy.
- **Empirical Validation:** PPM requires testing against experimental data to confirm its superiority over PEMDAS.

6 Conclusion

Parallel Permutation Mathematics offers a novel probabilistic method for modeling non-linear systems by evaluating all permutations of operation order and aggregating results with frequency-based weighting. The preprocessing algorithm reduces computational complexity to $O(k)$, where k is the number of equivalence classes, and modern tools like SymPy and multithreaded CPUs enable practical implementation today. The test case demonstrates PPM’s ability to capture multiple outcomes, suggesting applications in complex systems. Future work includes:

- Implementing the algorithm in a computational framework with optimized parallelization.
- Testing PPM against experimental data in chemistry or physics.
- Exploring context-specific weighting schemes to enhance accuracy.

Looking forward, advances in parallel hardware, such as Graphics Processing Units (GPUs), could further enhance PPM’s scalability. For instance, a system with a single CPU handling input parsing and result aggregation, paired with GPUs computing permutations in parallel (e.g., 20 GPUs for large expressions), could reduce evaluation time to that of a single PEMDAS computation. As CPU multithreading technology improves, standard computers may eventually handle all necessary permutations without additional hardware, making PPM even more accessible for real-world applications.

Acknowledgements

The author acknowledges the use of Grok, an artificial intelligence tool created by xAI, for assistance in drafting and structuring the LaTeX manuscript, polishing text for clarity and academic style, generating the TikZ code for Figure ??, suggesting references relevant to the applications of Parallel Permutation Mathematics, refining the discussion on computational feasibility with symbolic algebra tools and parallel hardware, and incorporating a two-number test case with 264 unique expressions. All content was generated under the author’s direction, with iterative feedback to ensure alignment with the author’s original ideas and mathematical contributions. The core concept, test case, and scientific decisions were made by the author.

7 References

References

- [1] Henrion, M. (1998). Probabilistic arithmetic. *Computers & Mathematics with Applications*, 36(5), 1–15.
- [2] Drineas, P., & Mahoney, M. W. (2016). Randomized algorithms for matrices and data. *Foundations and Trends in Machine Learning*, 8(3-4), 275–397.
- [3] Fitting, M. (1991). Many-valued logics. *Theoretical Computer Science*, 87(2), 237–254.
- [4] Atkins, P., & de Paula, J. (2010). *Physical Chemistry*. Oxford University Press.
- [5] Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press.
- [6] Strogatz, S. H. (2018). *Nonlinear Dynamics and Chaos*. CRC Press.